

Relazione di progettazione e scelte tecnologiche

Dall'interpretazione del JSON alla definizione dell'architettura full stack

Repository ufficiale

github.com/Emilio-Moscatiello/Perimetra-brief

Data della relazione: 13 luglio 2026

Razionale ricostruito dal brief, dal dataset e dal codice sorgente

Premessa e finalità del documento

Questa relazione descrive il percorso progettuale di Perimetra: come il brief iniziale e il file JSON siano stati interpretati, quali problemi informativi siano emersi e perché il codice sia stato organizzato in un frontend React e TypeScript, una logica dati Python condivisa, un server FastAPI per lo sviluppo e funzioni serverless per la pubblicazione su Vercel. L'attenzione non è rivolta all'elenco completo delle funzionalità, già trattato nella documentazione tecnica, ma alle ragioni delle scelte.

Le motivazioni sono ricostruite soltanto quando risultano coerenti e verificabili nel brief, nella struttura dei file e nel comportamento del codice. Quando viene discussa un'alternativa, essa rappresenta una valutazione progettuale ragionata, non la cronaca di una decisione personale documentata.

Oggetto della progettazione

Il compito iniziale chiedeva di elaborare un report JSON relativo alla sicurezza di un dominio dimostrativo e di renderlo consultabile attraverso una pagina HTML. Indicava una preferenza per TypeScript con React e, se necessario, Python per il backend; suggeriva grafici interattivi, filtri, sezioni di dettaglio e la simulazione di un'API. Richiedeva infine la pubblicazione del codice in un repository Git e l'hosting dell'applicazione.

La progettazione doveva quindi risolvere contemporaneamente un problema informativo, uno di interazione e uno di distribuzione. Il JSON era ricco di valori aggregati ma poco adatto alla consultazione diretta. L'interfaccia doveva trasformare numeri eterogenei in una gerarchia comprensibile. L'architettura doveva infine funzionare sia in locale sia su una piattaforma gratuita, senza introdurre infrastrutture sproporzionate rispetto alla natura dimostrativa del progetto.

La relazione procede dalla lettura del dataset alla definizione dei requisiti, quindi motiva modello dei dati, generazione sintetica, architettura, API, frontend, stato, visualizzazioni, stile, strumenti di sviluppo, backend Python e deployment. Si conclude valutando compromessi, limiti e possibili evoluzioni.

Lettura e interpretazione del JSON

La forma della sorgente

Il punto di partenza è `data/summary_seed.json`. La radice contiene `status` e un array `results`; il primo e unico elemento dell'array rappresenta il report di `cybersonar.demo`. Questa forma suggerisce la risposta di un servizio remoto: lo stato descrive l'esito della chiamata e `results` lascia aperta la possibilità di restituire più risultati. Il progetto conserva tale forma nelle risposte del riepilogo, anche se internamente lavora con il singolo oggetto.

Il report combina campi scalari, testi lunghi e strutture annidate. Identificativo, dominio e date descrivono il documento; `risk_score` e nove punteggi specifici esprimono valutazioni sintetiche; `n_port` associa porte a conteggi; `n_vulns` e `n_dataleak` rappresentano aggregati multidimensionali; `email_security`, `waf` e `cdn` aggiungono aspetti specialistici. La presenza di un riepilogo sia italiano sia inglese introduce infine un requisito di presentazione bilingue.

Il problema degli aggregati

La lettura del JSON fa emergere una distinzione decisiva. Il file sa quante vulnerabilità esistono per gravità e stato, ma non contiene le singole vulnerabilità. Sa quanti leak appartengono a ogni categoria, ma non offre una fonte, un identificativo o una data per ciascun evento. Indica tredici domini simili, senza elencarli. Fornisce il rischio corrente, ma non una serie storica. Pertanto il dataset è sufficiente per una panoramica, ma non per tutte le funzionalità di dettaglio suggerite dal brief.

La soluzione progettuale non poteva presentare dettagli inesistenti come se fossero contenuti nel file. Sono state quindi separate due responsabilità: preservare i valori originari e costruire dati dimostrativi aggiuntivi dichiaratamente sintetici. Tale separazione è implementata in `api/_data.py` e documentata nell'interfaccia e nella relazione tecnica. È una scelta importante perché permette di valorizzare la demo senza alterare il significato del seed.

Tipi e anomalie da gestire

`risk_score` è memorizzato come stringa, mentre gli altri punteggi sono numeri. Il frontend lo converte esplicitamente quando deve confrontarlo o rappresentarlo. Le categorie delle vulnerabilità e dei leak sono chiavi note, non array di oggetti omogenei. La stringa `spoofable` contiene una frase in inglese invece di un booleano; il codice la interpreta ricercando "possible". Questi aspetti hanno suggerito la creazione di contratti TypeScript espliciti anziché l'uso indiscriminato di oggetti non tipizzati.

Decisione progettuale: trattare il JSON come sorgente immutabile del report principale e costruire uno strato di adattamento. In questo modo le particolarità del formato rimangono isolate, mentre il frontend riceve contratti stabili.

Dai dati ai requisiti dell'interfaccia

Definizione della gerarchia informativa

Un singolo schermo contenente tutti i campi avrebbe prodotto una dashboard affollata e difficile da leggere. I dati sono stati quindi raggruppati per domanda dell'utente. La panoramica risponde a "quanto è rischioso il dominio e perché"; la pagina vulnerabilità approfondisce gravità e stato; la sezione leak tratta esposizione e risoluzione; rete e porte descrive la superficie tecnica; certificati ed email riunisce fiducia TLS, blacklist e spoofing; domini simili affronta il rischio di typosquatting.

Questa suddivisione motiva le sei route definite in `App.tsx`. Le pagine specialistiche non sono semplici copie del JSON: trasformano strutture diverse in interazioni coerenti. Vulnerabilità e leak richiedono filtri multipli e ricerca; rete richiede ordinamento e associazione porta-servizio; certificati richiede confronto; domini simili richiede una tabella compatta. La progettazione dei componenti comuni nasce da tali ripetizioni controllate.

Progressive disclosure

Il principio usato è mostrare prima la sintesi e rendere raggiungibile il dettaglio. Il punteggio complessivo e i KPI dominano la panoramica; barre e meter spiegano la composizione; i link conducono alle viste complete. Questo riduce il carico cognitivo e consente a utenti con competenze diverse di fermarsi al livello necessario. La stessa logica giustifica la presenza del riepilogo esecutivo bilingue in fondo alla pagina: offre una lettura narrativa dopo quella visuale.

Requisiti trasversali

Il cambio di dominio deve aggiornare tutte le sezioni; il tema deve rimanere coerente; caricamenti ed errori devono essere riconoscibili; l'interfaccia deve adattarsi a schermi differenti. Questi requisiti non appartengono a una singola pagina e hanno portato a introdurre uno stato globale limitato, componenti di feedback riutilizzabili e token CSS centralizzati. L'obiettivo non era creare un design system generale, ma evitare che ogni vista risolvesse nuovamente gli stessi problemi.

Strategia dei dati dimostrativi

Perché aggiungere contenuti sintetici

Il brief autorizzava l'uso di dati generati quando il JSON non fosse sufficiente e incoraggiava funzionalità aggiuntive capaci di mostrare competenze full stack. Limitarsi agli aggregati avrebbe escluso serie temporale, tabelle filtrabili e confronto fra profili di rischio. La generazione sintetica risponde quindi a un requisito dimostrativo concreto: rendere osservabili stati e interazioni che il seed, da solo, non poteva alimentare.

La scelta non è stata quella di produrre nuovi numeri a ogni richiesta. Un comportamento completamente casuale avrebbe fatto cambiare report durante la navigazione, rendendo difficile verificare l'applicazione e facendo apparire incoerenti grafici e conteggi. `random.Random` viene perciò inizializzato con una stringa composta da dominio e tipo di risorsa. Il risultato rimane pseudo-casuale ma ripetibile: una proprietà utile per demo, debugging e presentazioni.

Coerenza con gli aggregati

Per `cybersonar.demo` le righe di vulnerabilità rispettano esattamente quantità, severità e ripartizione attiva o passiva del JSON. Titoli, asset e date sono inventati, ma la tabella non contraddice i totali. I campioni di leak rispettano categorie e stato; il numero di righe è limitato a venticinque per categoria per non creare una tabella enorme, mentre i conteggi complessivi restano quelli originari. La cronologia forza l'ultimo punto a coincidere con il rischio corrente.

Profili aggiuntivi

Due domini interamente sintetici mostrano come l'interfaccia reagisca a posture differenti. `northwind-retail.demo` ha rischio guida 47 e profilo "Attenzione"; `aurora-fintech.demo` ha rischio 12 e profilo "Solido". I valori secondari sono generati attorno a tali livelli mediante distribuzioni gaussiane e limiti. L'uso di profili differenziati consente di verificare colori, soglie, messaggi e grafici senza modificare manualmente il seed.

Compromesso: i dati deterministici rendono la demo stabile, ma non diventano dati reali. Il codice deve continuare a dichiarare la loro natura simulata e non può essere usato come motore di valutazione della sicurezza.

Scelta dell'architettura full stack

Separazione fra presentazione, trasporto e dominio

Il repository separa il frontend, lo strato HTTP e la logica dei dati. React non legge direttamente il file JSON: chiama API relative. FastAPI e gli handler Vercel non contengono gli algoritmi di generazione: importano `api/_data.py`. Questa struttura riduce l'accoppiamento. Il dataset può cambiare senza riscrivere i componenti; il backend locale può essere sostituito senza modificare il client, purché mantenga il contratto; l'interfaccia può evolvere indipendentemente dalla generazione.

Un'unica logica per due ambienti

Il progetto deve essere comodo da sviluppare e semplice da distribuire. FastAPI offre in locale reload, validazione e documentazione automatica; Vercel preferisce funzioni serverless leggere. Duplicare il comportamento nei due ambienti avrebbe creato il rischio di risultati diversi. La decisione è stata condividere `_data.py` e mantenere distinti soltanto gli adattatori HTTP.

`server/app/main.py` espone le route FastAPI. I file pubblici in `api` definiscono invece handler basati sulla libreria standard. Entrambi richiamano `list_domains`, `get_summary`, `get_history`, `get_vulnerabilities`, `get_dataleaks` e `get_similar_domains`. Questa è la scelta architetturale centrale del backend: la piattaforma di esecuzione non determina le regole dei dati.

Assenza di database

Un database non è necessario per una demo alimentata da un seed statico e da dati ricostruibili. Introdurlo avrebbe richiesto schema, migrazioni, credenziali, hosting e gestione dello stato, senza risolvere un requisito del brief. I report vengono quindi costruiti in memoria durante l'importazione. Il compromesso è l'assenza di persistenza: una futura cronologia reale o la gestione di utenti richiederebbe un livello dati permanente.

Decisione progettuale: mantenere l'architettura minima capace di dimostrare il flusso full stack, evitando infrastrutture che non aggiungono valore alla consegna.

Perché React, TypeScript e React Router

React per la composizione dell'interfaccia

La dashboard contiene molte unità ripetute e dipendenti dallo stato: KPI, meter, barre, tabelle, filtri, skeleton e navigazione. React consente di descriverle come componenti e di aggiornare automaticamente la vista quando cambia dominio o arrivano nuovi dati. Una soluzione HTML e JavaScript imperativa sarebbe stata possibile, ma avrebbe richiesto gestione manuale del DOM, maggiore duplicazione e più attenzione alla sincronizzazione fra controlli e contenuto.

Gli hook nativi coprono le esigenze del progetto. `useState` gestisce filtri e preferenze; `useEffect` coordina richieste e persistenza; `useMemo` evita di ricalcolare inutilmente dati filtrati e serie; `useCallback` stabilizza le operazioni condivise; `useRef` collega il grafico al contenitore. Non è stata necessaria una libreria di stato esterna.

TypeScript come contratto

Il JSON presenta strutture annidate e chiavi categoriali precise. TypeScript permette di rappresentare severità, categorie e stati come unioni letterali e di descrivere report e risposte con interfacce. Questo riduce errori quali l'uso di una categoria inesistente, l'accesso a un campo con nome errato o la confusione fra oggetto e array. La modalità `strict` e il type-check prima della build rendono il contratto parte del processo di qualità.

TypeScript non valida i dati ricevuti a runtime. La scelta è adeguata a una demo in cui frontend e backend sono nello stesso repository, ma una soluzione produttiva dovrebbe aggiungere schemi runtime o generare i tipi da un contratto OpenAPI. La relazione fra vantaggio e limite è importante: la tipizzazione migliora lo sviluppo, non sostituisce la validazione delle risposte.

React Router per una SPA navigabile

Le sezioni tematiche devono avere URL distinti e mantenere sidebar e barra superiore. React Router DOM risolve questo requisito senza ricaricare il documento. `BrowserRouter` gestisce la cronologia, `Routes` associa percorsi a pagine, `NavLink` segnala la voce attiva e `Link` collega panoramica e dettagli. Il rewrite Vercel verso `index.html` completa la scelta lato hosting.

Perché non introdurre Redux: lo stato globale è ridotto a dominio, report, tema e feedback di caricamento. Context e hook nativi sono sufficienti e mantengono bassa la complessità.

Progettazione dello stato e del client API

AppContext come coordinatore

Il dominio selezionato influenza tutte le pagine e il report è mostrato anche nella barra superiore. Questi dati appartengono quindi a uno stato condiviso. `AppProvider` carica i domini, recupera il riepilogo e distribuisce dati, caricamento, errore e tema. La persistenza nel `localStorage` migliora la continuità fra sessioni senza richiedere account o backend.

Le risorse specialistiche non sono inserite nel Context. Cronologia, vulnerabilità, leak e domini simili vengono richiesti dalle pagine che li usano. Questa separazione evita di caricare tutto all'avvio e mantiene il provider concentrato sullo stato realmente globale. L'hook `useResource` uniforma la sequenza caricamento-risultato-errore e impedisce aggiornamenti dopo lo smontaggio mediante un flag di cancellazione logica.

Client HTTP centralizzato

`frontend/src/api/client.ts` concentra URL, codifica dei parametri e gestione degli errori. I componenti non costruiscono richieste direttamente e non conoscono la posizione del backend. La costante `BASE` vale `/api`, scelta che rende lo stesso bundle compatibile con proxy Vite e funzioni Vercel. `encodeURIComponent` protegge la struttura della query quando viene inserito il dominio.

L'helper generico `getJSON` associa la risposta attesa a un tipo TypeScript. Se lo stato HTTP non è positivo, prova a leggere un messaggio JSON e altrimenti usa il codice. Questo comportamento è sufficiente per la demo, anche se le differenze fra errori FastAPI e serverless suggeriscono una futura normalizzazione.

Persistenza delle preferenze

Il tema e il dominio non sono informazioni sensibili né condivise fra dispositivi; `localStorage` è quindi proporzionato al requisito. Se non esiste una preferenza del tema, `matchMedia` rispetta quella del sistema operativo. L'attributo `data-theme` sull'elemento radice permette al CSS di applicare l'intera palette senza modificare i componenti.

Visualizzazioni senza librerie di charting

Motivazione della scelta

Il progetto utilizza soltanto React, React DOM e React Router come dipendenze runtime dirette. Gauge, andamento temporale, barre orizzontali e barre impilate sono costruiti con SVG, HTML e CSS. Per un insieme limitato di grafici, questa scelta riduce il bundle, evita una dipendenza complessa e dimostra la capacità di trasformare valori in geometrie e interazioni.

Una libreria come Recharts, Chart.js o D3 avrebbe accelerato alcune funzionalità, ma avrebbe introdotto API e stili esterni per problemi relativamente semplici. D3 sarebbe stato potente ma sovradimensionato; una libreria canvas avrebbe richiesto ulteriore lavoro per accessibilità e responsività. La soluzione nativa mantiene il controllo sul linguaggio visuale di Perimetra.

Logica geometrica

Il gauge calcola la circonferenza del cerchio e ne usa il 75% come arco disponibile; il punteggio determina la porzione colorata. Il grafico temporale mappa l'indice del punto sull'asse orizzontale e il rischio su una scala verticale da zero a cento, costruendo un percorso SVG e un'area. `ResizeObserver` aggiorna la larghezza in base al contenitore. Le barre orizzontali normalizzano rispetto al massimo; quelle impilate normalizzano rispetto alla somma.

Interazione e accessibilità

Gli hover mostrano dettagli e modificano enfasi, mentre ruoli ed etichette ARIA descrivono gauge e meter. La resa SVG rimane nitida a qualsiasi densità. Il compromesso è che le interazioni avanzate, l'esportazione e la navigazione completa da tastiera nei grafici non sono fornite automaticamente. Un prodotto maturo dovrebbe verificare tali aspetti con test di accessibilità.

Decisione progettuale: implementare direttamente le visualizzazioni necessarie, mantenendo il codice leggibile e il numero di dipendenze contenuto.

Componenti, filtri e organizzazione del frontend

Componenti riutilizzabili

`DataTable` accetta colonne generiche, righe e chiave, così tre pagine possono condividere struttura e comportamento. `StatTile` standardizza i KPI; `Meter`, `StackedBar` e `HorizontalBarChart` separano il calcolo visuale dal dominio che fornisce i dati. `LoadingCard` ed `ErrorState` rendono prevedibili gli stati asincroni. Questa composizione riduce duplicazioni senza creare un'astrazione eccessivamente generale.

Filtri lato client

Gli endpoint restituiscono dataset piccoli o campionati. È quindi ragionevole filtrare nel browser con `useMemo`: la risposta è immediata e non richiede una chiamata a ogni selezione o carattere digitato. Severità, stato e testo vengono combinati in predicati leggibili. Per dataset operativi molto grandi, la stessa scelta diventerebbe inefficiente e andrebbe sostituita con paginazione e filtri server-side.

Tipi separati dalla resa

Le interfacce in `types/report.ts` descrivono il dominio informativo; `lib/colors.ts` traduce severità e categorie in etichette e colori; le pagine scelgono come comporre i componenti. Questa separazione evita di inserire traduzioni, soglie e palette direttamente in ogni vista. Il codice rimane abbastanza semplice da seguire e, allo stesso tempo, localizza le regole condivise.

Scelte di stile e accessibilità

Identità visuale

La dashboard adotta un'estetica da dossier operativo: fondo a griglia, superfici nette, sidebar scura e accento ciano. Questa scelta rende riconoscibile il contesto cybersecurity senza affidarsi a immagini o asset esterni. `tokens.css` centralizza superfici, testo, bordi, serie, stati e ombre; `global.css` applica layout e componenti.

Token e temi

I colori non sono distribuiti casualmente nei componenti, ma richiamati tramite variabili CSS semantiche. Lo stesso stato "critical" può cambiare tinta fra tema chiaro e scuro senza modificare React. Le palette esplicite per entrambi i temi evitano semplici inversioni che potrebbero compromettere leggibilità e gerarchia. La persistenza del tema è gestita fuori dal CSS, mentre la resa rimane interamente dichiarativa.

Responsive e riduzione del movimento

Le media query trasformano la sidebar in navigazione orizzontale, riducono le griglie e adattano filtri e spaziature. `prefers-reduced-motion` limita animazioni e transizioni. Focus visibili, etichette accessibili e ruoli semantici dimostrano attenzione all'uso da tastiera e alle tecnologie assistive. Non sono tuttavia presenti audit automatici o test con screen reader: l'accessibilità è un obiettivo progettuale parziale, non una conformità certificata.

Perché Python, FastAPI e Uvicorn

Python per l'elaborazione

Il brief esprimeva una preferenza per Python nel backend. Il linguaggio si presta alla lettura del JSON, alla manipolazione di dizionari, alla generazione di dati e alla costruzione di date con poco codice cerimoniale. I moduli standard sono sufficienti per quasi tutta la logica: `json`, `random`, `hashlib`, `datetime` e `pathlib`. Non è stata introdotta una libreria di elaborazione dati come pandas perché i volumi e le trasformazioni non la giustificano.

FastAPI come ambiente locale

FastAPI permette di dichiarare route e query in modo conciso, con validazione automatica e codici di errore chiari. La documentazione OpenAPI su `/docs` e `/redoc` rende immediata l'ispezione degli endpoint. Le annotazioni Python si integrano con il framework e la struttura resta leggibile. Flask avrebbe potuto svolgere il ruolo, ma avrebbe richiesto più lavoro manuale per validazione e documentazione.

Uvicorn esegue l'applicazione ASGI e, in sviluppo, può osservare le modifiche. L'extra `standard` installa componenti consigliati dal server. FastAPI e Uvicorn sono dichiarati in `server/requirements.txt` perché appartengono alla modalità locale; le funzioni di produzione non ne dipendono.

CORS aperto nella demo

Il server accetta tutte le origini per semplificare l'integrazione. Vite usa comunque un proxy same-origin, ma CORS aperto rende gli endpoint facilmente testabili. È una scelta adatta a una demo pubblica di sola lettura; in produzione reale dovrebbe essere sostituita da origini consentite, autenticazione e protezioni contro abuso.

Perché funzioni serverless e Vercel

Coerenza con la consegna

Il brief richiede un link immediatamente utilizzabile e suggerisce Vercel. La piattaforma ospita bene un frontend Vite statico e consente di affiancare funzioni Python senza amministrare un server persistente. Per una demo a traffico ridotto, il modello serverless limita la configurazione e collega direttamente il deploy al repository GitHub.

Handler basati sulla libreria standard

I file in `api` usano `BaseHTTPRequestHandler` attraverso `JSONHandler`. Questa scelta mantiene leggere le funzioni e separa FastAPI dal runtime produttivo. L'handler comune gestisce query, serializzazione JSON, CORS, preflight ed errori. Ogni endpoint pubblico rimane un adattatore minimo intorno alle funzioni di `_data.py`.

Configurazione Vercel

`vercel.json` dichiara il comando di build, la directory `frontend/dist`, il runtime Python 3.12, l'inclusione del dataset e il rewrite SPA. La configurazione esplicita riduce le impostazioni manuali. L'inclusione di `data/**` è essenziale: senza di essa le funzioni non troverebbero il seed nel bundle.

Compromessi del serverless

Le funzioni possono avere cold start e non conservano stato affidabile fra invocazioni. Nel progetto ciò non rappresenta un problema, perché i dati sono statici e ricostruibili. Una futura applicazione con scansioni lunghe, code o elaborazioni intensive potrebbe richiedere servizi persistenti diversi. Inoltre la validazione serverless non coincide perfettamente con FastAPI: uniformarla sarebbe una priorità prima di un uso produttivo.

Vite, build e dipendenze di sviluppo

Vite come ambiente frontend

Vite offre avvio rapido, trasformazione TSX, hot module replacement e build ottimizzata. La configurazione registra il plugin React, l'alias `@`, la porta 5173 e il proxy `/api`. L'alias rende gli import indipendenti dalla profondità delle cartelle; il proxy consente al browser di usare un'unica origine locale e mantiene identici gli URL fra sviluppo e produzione.

Plugin e tipi

`@vitejs/plugin-react` integra JSX e aggiornamento React nel flusso Vite. `@types/react` e `@types/react-dom` forniscono al compilatore le dichiarazioni delle librerie JavaScript. Sono dipendenze di sviluppo perché non devono essere caricate nel browser. TypeScript viene eseguito con `noEmit`: Vite produce il bundle, mentre `tsc` controlla soltanto i tipi.

Build come controllo

Lo script `build` concatena type-check e bundling. Se TypeScript trova un'incoerenza, Vite non parte; in caso contrario produce gli artefatti in `dist`. Questa sequenza trasforma il compilatore in una barriera minima prima del deploy. La build è stata verificata con successo sul repository analizzato.

Dipendenze non necessarie e limite del lint

Non sono state aggiunte librerie UI, client HTTP, charting o gestione dello stato perché le API native soddisfano i requisiti. Il lockfile conserva dipendenze transitive di Vite e React Router, ma non sono scelte dirette dell'applicazione. `package.json` dichiara uno script `lint` che richiama ESLint, tuttavia pacchetto e configurazione mancano: la progettazione della qualità è quindi incompleta e dovrebbe essere conclusa con lint e test automatici.

Alternative, compromessi e limiti delle decisioni

Un solo backend invece di due adattatori

Si sarebbe potuto distribuire direttamente FastAPI su un servizio capace di eseguire un server Python persistente. La soluzione avrebbe uniformato errori e validazione, ma avrebbe aggiunto un secondo provider o una configurazione diversa da Vercel. L'architettura attuale privilegia semplicità di deploy e dimostra capacità serverless, al costo di mantenere due sottili strati HTTP.

Dati statici direttamente nel frontend

Il JSON avrebbe potuto essere importato da React e trasformato nel browser. Questa alternativa sarebbe stata più semplice, ma non avrebbe mostrato una separazione full stack, avrebbe esposto la generazione nel bundle e reso meno naturale una futura sostituzione con dati reali. L'API simulata offre un contratto più vicino a un'applicazione reale.

Libreria grafica esterna

Una libreria avrebbe fornito più tipi di grafico e interazioni pronte. Perimetra richiede però poche forme specifiche e una forte coerenza visuale. L'implementazione nativa riduce dipendenze e aumenta controllo, ma richiede manutenzione e test accessibilità. La scelta rimane valida finché il numero e la complessità dei grafici sono limitati.

Stato globale più strutturato

Redux, Zustand o una libreria di server state potrebbero gestire cache, deduplicazione e invalidazione. Nel progetto corrente avrebbero introdotto concetti aggiuntivi per poche richieste e uno stato semplice. Se l'applicazione acquisisse autenticazione, mutazioni, molte risorse o caching avanzato, la valutazione cambierebbe.

Limiti riconosciuti

L'assenza di test, lint operativo, validazione runtime e persistenza riduce l'affidabilità oltre la demo. CORS è permissivo, non esistono autenticazione o autorizzazione e i dati non sono reali. Questi limiti non invalidano le scelte per l'obiettivo dimostrativo, ma definiscono chiaramente il confine oltre il quale sarebbe necessaria una nuova fase progettuale.

Valutazione conclusiva

La progettazione di Perimetra può essere letta come un esercizio di proporzione: trasformare un input aggregato in un'esperienza full stack completa, senza far passare la simulazione per un sistema operativo reale e senza introdurre tecnologie non necessarie.

La lettura del JSON ha determinato la struttura informativa e ha evidenziato i dati mancanti. La separazione fra originari, derivati e sintetici ha permesso di costruire viste ricche mantenendo trasparenza. React e TypeScript hanno dato forma a un'interfaccia componentizzata e tipizzata; Context e hook hanno gestito uno stato limitato senza librerie aggiuntive; SVG e CSS hanno reso possibili visualizzazioni coerenti senza un framework grafico.

Python ha concentrato caricamento e generazione in una logica leggibile. FastAPI ha migliorato l'esperienza locale, mentre gli handler serverless hanno reso la distribuzione compatibile con Vercel. Vite ha unificato sviluppo, proxy e build. Il risultato è un'architettura sufficientemente realistica da dimostrare competenze full stack, ma abbastanza semplice da rimanere comprensibile e distribuibile.

Le scelte non sono definitive per ogni scala. Un prodotto reale richiederebbe sorgenti verificate, database, contratti runtime, autenticazione, test, CI, osservabilità e controlli di sicurezza. Nel perimetro del brief, tuttavia, la soluzione mantiene un buon equilibrio fra chiarezza, valore dimostrativo, riuso e facilità di consegna.

Il repository ufficiale è disponibile all'indirizzo <https://github.com/Emilio-Moscatiello/Perimetra-brief>. L'URL per la clonazione è <https://github.com/Emilio-Moscatiello/Perimetra-brief.git>.

Relazione redatta sulla base di `data/project_brief.txt`, `data/summary_seed.json`, configurazioni e codice sorgente del repository locale. Le alternative discusse costituiscono analisi progettuale e non affermazioni su decisioni storiche non documentate.